Case Studies: ResNets & Batch Normalization

Deep Convolutional Models

Firuz Kamalov

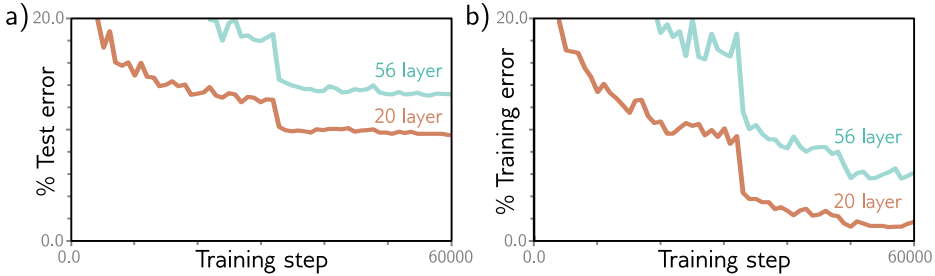
MAI 604: Deep Learning

April 24, 2026

Lecture Outline

The Problem with Very Deep Network

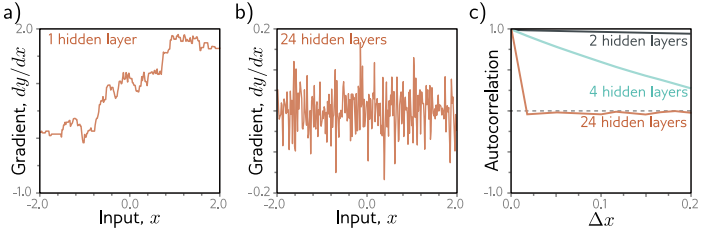
Image classification performance improved as the depth of CNNs was extended from 8 layers (AlexNet) to 19 layers (VGG). However, performance decreased again when more layers were added.



The Problem with Very Deep Network

Root Cause

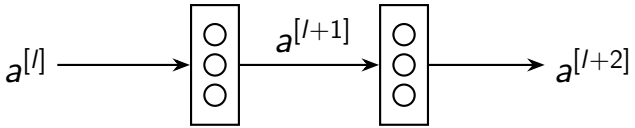
- Vanishing gradients: Gradient signals become infinitesimal as they propagate back to earlier layers.
- Exploding gradients: Gradient signals become overly large, causing numerical instability.
- Shattered gradients: Gradient changes very quickly and unpredictably

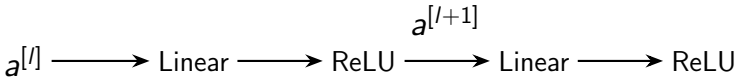


Solution: Residual Networks (ResNets) introduced by Kaiming He et al. (2015).
Core Idea: Use *skip connections* (shortcut) to pass activations from one layer to a much deeper layer, bypassing intermediate steps.

Standard Neural Network Layer (Main Path)

Consider a standard neural network (main path), where information flows sequentially.



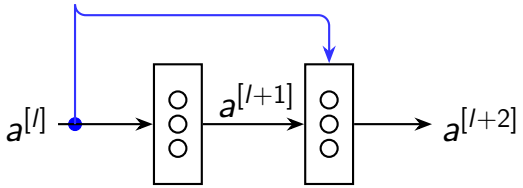


$$z^{[l+1]} = W^{[l+1]} a^{[l]} + b^{[l+1]}$$
$$a^{[l+1]} = g(z^{[l+1]})$$

$$z^{[l+2]} = W^{[l+2]} a^{[l+1]} + b^{[l+2]}$$
$$a^{[l+2]} = g(z^{[l+2]})$$

Adding the Skip Connection (Residual Block)

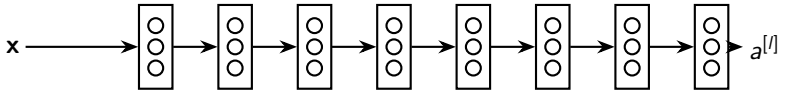
In a **Residual Block**, we add a **"short cut" / skip connection** that fast-forwards $a^{[l]}$ directly to the input of the second layer's activation function.



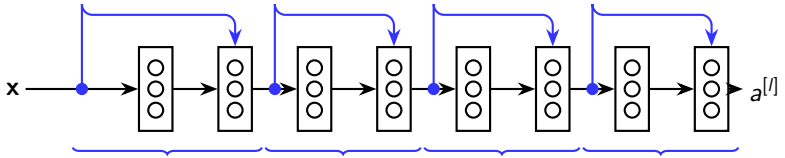
$$a^{[l+2]} = g(z^{[l+2]} + a^{[l]}) \quad (\textit{replaces } a^{[l+2]} = g(z^{[l+2]}))$$

Network Architectures: Plain vs. ResNet

"Plain Network" (Layers are simply stacked sequentially without shortcuts)



Residual Network (ResNet) (Skip connections added every 2 layers)



Numerical Example: Residual Block Calculation

1. Initial Input

Let our starting activation be:

$$a^{[l]} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

2. First Layer ($l+1$)

Assume weights $W^{[l+1]} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$, $b^{[l+1]} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$.

$$\begin{aligned} z^{[l+1]} &= W^{[l+1]} a^{[l]} + b^{[l+1]} \\ &= \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 \\ -2 \end{bmatrix} \\ a^{[l+1]} &= \text{ReLU}\left(\begin{bmatrix} 1 \\ -2 \end{bmatrix} \right) = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \end{aligned}$$

3. Second Layer ($l+2$)

Assume $W^{[l+2]} = \begin{bmatrix} 2 & 1 \\ -1 & 1 \end{bmatrix}$, $b^{[l+2]} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$.

$$\begin{aligned} z^{[l+2]} &= W^{[l+2]} a^{[l+1]} + b^{[l+2]} \\ &= \begin{bmatrix} 2 & 1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} -1 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 2 \\ -1 \end{bmatrix} + \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \end{aligned}$$

Plain Network Output:

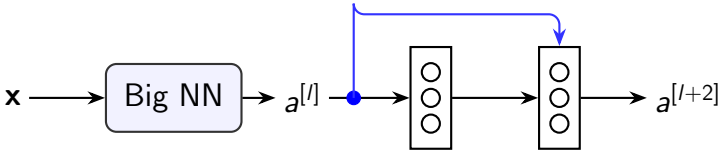
$$a^{[l+2]} = \text{ReLU}(z^{[l+2]}) = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

ResNet Output:

$$a^{[l+2]} = \text{ReLU}\left(z^{[l+2]} + a^{[l]}\right)$$

$$a^{[l+2]} = \text{ReLU}\left(\begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 \\ 2 \end{bmatrix} \right) = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

Why ResNets Work: Learning the Identity Function



Mathematical Justification:

$$a^{[l+2]} = g\left(z^{[l+2]} + a^{[l]}\right)$$
$$= g\left(\underbrace{W^{[l+2]}a^{[l+1]} + b^{[l+2]}}_{=0 \text{ if } W\approx 0, b\approx 0} + a^{[l]}\right)$$

- L2 regularization (weight decay) pushes weights $W^{[l+2]}$ and biases $b^{[l+2]}$ towards zero.

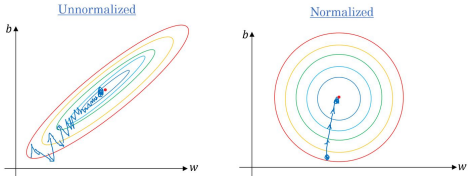
$$a^{[l+2]} \approx g\left(a^{[l]}\right)$$
$$= a^{[l]} \quad (\text{since } a^{[l]} \geq 0 \text{ due to ReLU})$$

Key Insight

It is easy for a residual block to learn the **identity function**. Thus, adding it to a "Big NN" guarantees performance won't degrade, while maintaining the potential to learn complex features and improve!

The Need for Batch Normalization

- **Input Normalization:** Scaling input features speeds up learning by making loss contours symmetric.
- **The Deep Network Problem:** As earlier layer weights update during training, the distributions of intermediate activations ($z^{[l]}$) shift constantly (internal covariate shift).
- It forces later layers to continuously adapt to changing distributions, slowing down training.
- **Solution:** Normalize the *intermediate* values ($z^{[l]}$) in the hidden layers!



Normalizing inputs makes optimization smoother and faster.

Batch Normalization: The Equations

Given some intermediate values in a Neural Network for layer l across a mini-batch of size m. Let $z_i = z_i^{[l]}$ represent the value for the i-th example:

Step 1: Compute Batch Mean and Variance

$$\mu = \frac{1}{m} \sum_{i=1}^{m} z_i$$

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^{m} (z_i - \mu)^2$$

Step 2: Normalize

$$z_{\text{norm},i} = \frac{z_i - \mu}{\sqrt{\sigma^2 + \varepsilon}} \quad (\varepsilon \text{ is a small constant for numerical stability})$$

Step 3: Scale and Shift

$$\tilde{z}_i = \gamma z_{\text{norm},i} + \beta$$

Note: γ and β are **learnable parameters** of the model, optimized via backpropagation just like weights and biases.

Numerical Example: Batch Normalization

Let's calculate Batch Normalization for a single neuron across a **mini-batch of size $m = 4$**.
Assume the pre-activation values for the 4 examples are: $z_1 = 1, z_2 = 1, z_3 = 7, z_4 = 7$.

1. Compute Mean (μ)

$$\mu = \frac{1}{m}\sum_{i=1}^{m} z_i = \mathbf{4}$$

2. Compute Variance (σ^2)

$$\sigma^2 = \frac{1}{m}\sum_{i=1}^{m}(z_i - \mu)^2 = \mathbf{9}$$

3. Normalize

$$z_{\text{norm},1} = \frac{z_1 - \mu}{\sigma} = \mathbf{-1}$$

$$z_{\text{norm},3} = \frac{z_3 - \mu}{\sigma} = \mathbf{1}$$

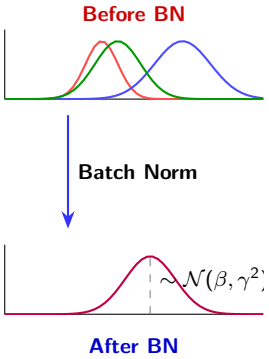
4. Scale and Shift ($\tilde{z}_i$)
Assume learned parameters are $\gamma = 2$ and $\beta = 10$.

$$\tilde{z}_i = \gamma z_{\text{norm},i} + \beta$$

Final Output to Activation Function:
$\tilde{z}_1 = 2(-1) + 10 = \mathbf{8}$
$\tilde{z}_3 = 2(1) + 10 = \mathbf{12}$

How Batch Normalization Fixes the Issue

- **Decouples Layers:** Stabilizes intermediate distributions. Later layers don't have to chase wildly shifting inputs.
- **Restores Power:** γ and β let the network learn the optimal scale and mean (e.g., to use the non-linear parts of an activation function).
- **Faster Optimization:** A smoother loss landscape allows the use of much **higher learning rates**.
- **Slight Regularization:** Calculating statistics on mini-batches adds slight noise, acting somewhat like dropout.



ResNet Block

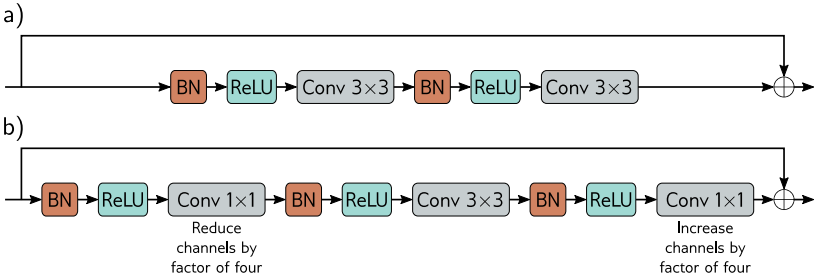


Figure: ResNet blocks. a) A standard block in ResNet contains batch normalization, followed by an activation, and a 3×3 conv layer. Then, this sequence is repeated. b). A *bottleneck* ResNet makes more efficient use of parameters due to 1×1 kernel.

ResNet

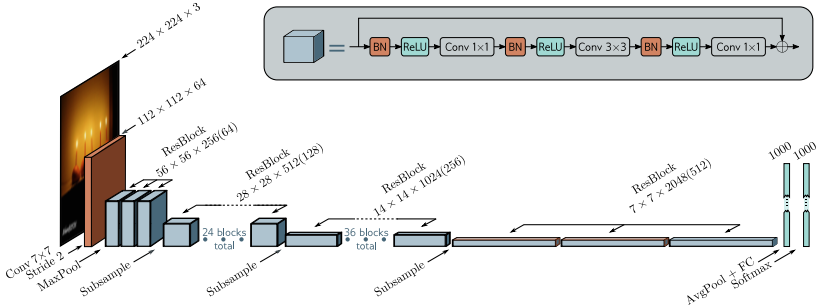


Figure: ResNet-200 model. Achieved top-5 error rate 4.6% which is better than VGG (6.8%) and human (5.1%) performance.

DenseNet

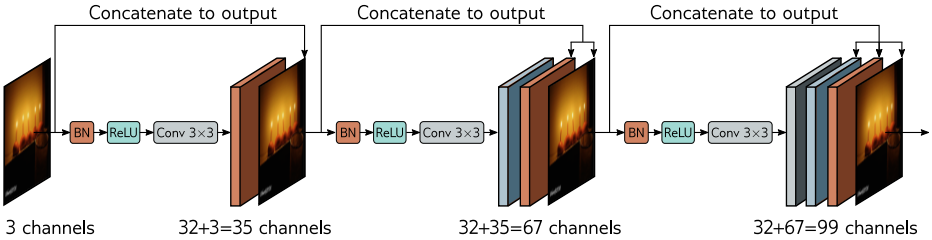


Figure: DenseNet. This architecture uses residual connections to concatenate the outputs of earlier layers to later ones. Here, the three-channel input image is processed to form a 32-channel representation. The input image is concatenated to this to give a total of 35 channels. This combined representation is processed to create another 32-channel representation, and both earlier representations are concatenated to this to create a total of 67 channels and so on.) performance.

Summary

- Standard deep neural networks suffer from **vanishing and exploding gradients**, making optimization highly challenging.
- **Residual Blocks** introduce a shortcut path ($a^{[l]}$ added to $z^{[l+2]}$), allowing gradients and information to bypass intermediate layers.
- Stacking these blocks creates a **ResNet**.
- Unlike Plain networks, ResNets do not degrade in training performance as depth increases, safely enabling models with hundreds of layers without loss of performance.